

CE 310

Python Tutorial — Simple Linear Regression

Week 9 · Session 2 · Hands-on Walkthrough

University of Arizona · Dr. Wooyoung Jung · Fall 2026

Today's Plan — Five New Skills, Then the In-Class Exercise

By the end of this walkthrough you will know how to:

1. Load a CSV and explore summary statistics with **pandas**
2. Compute Pearson correlation with `df['A'].corr(df['B'])`
3. Fit a regression model with `scipy.stats.linregress`
4. Plot a scatter plot with a regression line using **matplotlib**
5. Create a **residual plot** to validate the model

Assumed from the Quickstart: you can open Colab, run a cell, and understand `df['column']` syntax.

File needed: `Week5_D0E_Med0ffice_ZoneTemp_2023.csv` — upload to Colab before starting.

Step 1 — Setup: Import Libraries and Load Data

 Run this cell first — it must succeed before anything else works.

```
import pandas as pd
import matplotlib.pyplot as plt
from scipy import stats
import numpy as np

df = pd.read_csv('Week5_D0E_MedOffice_ZoneTemp_2023.csv')
print(f"Rows: {len(df)}")
print(df.columns.tolist())
```

Expected output: Rows: 8760

If you see `FileNotFoundError`, the CSV has not been uploaded yet. Use the Colab Files panel (folder icon on the left) → upload icon → select the CSV.

Step 2 — Explore the Data

Before fitting any model, always check the shape of your data.

```
print(df['Outdoor_Temp_F'].describe())
print(df['Cooling_kWh'].describe())
```

Expected key values:

Statistic	Outdoor_Temp_F	Cooling_kWh
mean	68.7°F	19.0 kWh/hr
min	26.6°F	0.0 kWh/hr
max	106.4°F	121.2 kWh/hr

`.describe()` prints count, mean, std, min, the three quartiles (25%/50%/75%), and max for a column — the same summary statistics you computed by hand in Weeks 1–2. The minimum cooling of 0.0 is important — the chiller turns off in winter. This will affect the residual plot later.

Step 3 — Correlation (Bridge from Week 7)

 **Compute Pearson r using pandas:**

```
r = df['Outdoor_Temp_F'].corr(df['Cooling_kWh'])  
print(f"Pearson r = {r:.4f}")
```

Expected: `Pearson r = 0.7752`

This matches the Excel `CORREL` result you computed in Week 7. The two methods are identical — pandas uses the same Pearson formula.

`df['A'].corr(df['B'])` always returns a value between -1 and $+1$. For covariance, use `.cov()` instead.

Step 4 — Fit the Regression with scipy

 One function call fits the model. ****x (predictor) goes first, y (outcome) goes second**** — swapping them gives the wrong slope.

```
slope, intercept, r_val, p_val, se = stats.linregress(
    df['Outdoor_Temp_F'],
    df['Cooling_kWh']
)
print(f"Slope:      {slope:.4f} kWh/°F")
print(f"Intercept:  {intercept:.4f} kWh")
print(f"r:           {r_val:.4f}")
print(f"R²:          {r_val**2:.4f}")
print(f"p-value:      {p_val:.2e}")
```

Step 4 — Interpreting the Output

Expected: slope = 1.1585 · intercept = -60.68 · $R^2 = 0.6010$

⚠ `r_val` is Pearson r , not R^2 . You must square it: `r_val**2`.

Step 5 — Model Fit Values

Return value	What it is	Expected value
<code>slope</code>	β_1 — change in $\hat{y}$ per 1°F	1.1585 kWh/°F
<code>intercept</code>	β_0 — predicted $\hat{y}$ at $x = 0$	−60.68 kWh
<code>r_val</code>	Pearson r (not R^2)	0.7752
<code>r_val**2</code>	R^2 — fraction of variance explained	0.6010

Step 5 — Significance & Interpretation

Return value	What it is	Expected value
<code>p_val</code>	Is this slope real or noise? $p \approx 0$ means yes	≈ 0.0 (real relationship)
<code>se</code>	Standard error of the slope	0.0101 kWh/°F

A p-value of 0.0 means the slope is statistically indistinguishable from zero with essentially zero probability — outdoor temperature is a real predictor.

Step 6 — Plot: Setup & Scatter

```
x = df['Outdoor_Temp_F']  
y_fit = slope * x + intercept  
  
fig, ax = plt.subplots(figsize=(8, 5))  
ax.scatter(x, df['Cooling_kWh'],  
           alpha=0.2, s=5, color='steelblue', label='Observed')
```

Step 6 — Plot: Regression Line & Labels

```
# sort x so the line draws left-to-right (not zigzag)
x_sorted = x.sort_values()
ax.plot(x_sorted, slope * x_sorted + intercept,
        color='red', linewidth=2,
        label=f' $\hat{y} = \{\text{slope}:.2f\}x + \{\text{intercept}:.1f\}$ ')
ax.set_xlabel('Outdoor Temperature (°F)')
ax.set_ylabel('Cooling Energy (kWh/hr)')
ax.set_title('Cooling Energy vs. Outdoor Temperature')
ax.legend()
plt.tight_layout()
plt.show()
```

The cloud of points fans out at higher temperatures — variance is not constant, which is one limitation of this model.

Step 7 — Residual Plot

```
residuals = df['Cooling_kWh'] - (slope * x + intercept)

fig, ax = plt.subplots(figsize=(8, 5))
ax.scatter(x, residuals, alpha=0.2, s=5, color='gray')
ax.axhline(0, color='red', linewidth=1.5, linestyle='--')
ax.set_xlabel('Outdoor Temperature (°F)')
ax.set_ylabel('Residual (kWh/hr)')
ax.set_title('Residual Plot — Cooling Model')
plt.tight_layout()
plt.show()
```

What to look for: at outdoor temperatures below ~45°F, all residuals are **positive**. The model predicts negative cooling (physically impossible), but actual values are 0 — so residual = 0 – (negative) > 0. This systematic pattern is the model's main structural flaw.

Step 8 — Make a Prediction

```
# CE track: predict at ASHRAE 1% cooling design temperature
design_temp = 108.0    # °F
pred_ce = slope * design_temp + intercept
print(f"Predicted cooling at {design_temp}°F: {pred_ce:.1f} kWh/hr")
print(f"Annual mean:
      f"{df['Cooling_kWh'].mean():.1f} kWh/hr")

# ArcE track: predict at economizer cutoff
econ_cutoff = 75.0    # °F (ASHRAE 90.1, Climate Zone 2B)
pred_ae = slope * econ_cutoff + intercept
print(f"Predicted cooling at {econ_cutoff}°F: {pred_ae:.1f} kWh/hr")
```

Expected: 64.4 kWh/hr at 108°F · 26.2 kWh/hr at 75°F

The 108°F design load is **3.4× the annual mean** — sizing for the average would leave the building severely undercooled.

Quick Check — Before You Start the In-Class Exercise

Verify your numbers, then open `Wk09.03_In_Class_Exercise.ipynb`

Confirm all three values before moving on:

- $r = 0.7752$ (matches Excel CORREL from Week 7)
- **slope** = 1.1585 kWh/°F
- $R^2 = 0.6010$

Question: Your residual plot shows all positive residuals at outdoor temperatures below ~45°F. What physical event causes every data point in that temperature range to have Cooling_kWh = 0?

Quick Reference — Week 9 New Python Tools

Task	Code
Column statistic	<code>df['A'].mean()</code> — also <code>.median()</code> , <code>.std()</code> , <code>.min()</code> , <code>.max()</code>
Pearson r	<code>df['A'].corr(df['B'])</code>
Fit regression	<code>stats.linregress(x, y)</code>
R ²	<code>r_val ** 2</code>
Predict at x*	<code>slope * x_star + intercept</code>
Residuals	<code>y - (slope * x + intercept)</code>

Task	Code
Scatter plot	<code>ax.scatter(x, y,</code> <code>alpha=0.2, s=5)</code>
Add fit line	<code>ax.plot(x_sorted, y_fit)</code>
Horizontal line	<code>ax.axhline(0, color='red')</code>
Axis labels	<code>ax.set_xlabel(...)</code>
Filter rows	<code>df[df['Season']=='Summer']</code>
Row count, count matching	<code>len(df), (df['A'] ></code> <code>0).sum()</code>

Before You Submit

⚠ Following this submission format exactly is essential — with a class this size, grading depends on it. Submissions that don't comply will be penalized.

1. **Run all cells first.** In Colab: Runtime → Run all. Wait until every cell shows a checkmark — no ⏳ spinners remaining.
2. **Download as .ipynb.** File → Download → Download .ipynb. Do not submit PDF or .py.
3. **Do not edit the `print` lines.** Lines like `print(f"ANSWER_A1 = {ANSWER_A1}")` must remain exactly as written — changing or deleting them means that part of your answer will not be counted.
4. **File name —** `CE310_W9_<NetID>.ipynb`, e.g., `CE310_W9_abc123.ipynb`.
5. **Upload** the `.ipynb` file to D2L Dropbox by **9:00 AM next Wednesday**.

You're Ready — Open the In-Class Exercise

`Wk09.03_In_Class_Exercise.ipynb` · **Upload:** `Week5_D0E_MedOffice_ZoneTemp_2023.csv`

- **Section A** (in-class): exploratory stats — rows, mean, std, min, max, median, r
- **Section B** (in-class + take-home): fit regression, interpret slope, compute R^2 , plot, predict

Take-home: B7 (scatter + fit line), B8 (residual plot), B9/B10 (design predictions), B11 (compare to Excel r), and the Memo.

Raise your hand if `linregress` returns a slope far from 1.16 — the most common cause is swapped x and y arguments.